FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES

CS054
(INTELLIGENT SYSTEMS)

EXERCISE

2

Sentiment Analysis

Name:	**BY PAIR^^**
Members:	**Dancel, Jamella Tracy L.** **Paulino, Armand George B.**
Section:	**AN42**
Professor:	**Ms. Crisola G. Tan**

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE •

Design and evaluate solutions for computing problems. [PO: e]

II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

• To design an Intelligent System based on methods learned. [CLO: 3]

III. INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Demonstrate understanding of Sentiment Analysis.
- Design and develop a simple program about Sentiment Analysis using Python.

IV. BACKGROUND INFORMATION

Sentiment Analysis can assist us in determining the mood and feelings of the general public as well as obtaining useful information about the setting. Sentiment Analysis is the process of assessing data and categorizing it according to the research's needs.

Sentiments can be used to gain a better understanding of different events and their consequences. It goes by various names, including "sentiment analysis," "opinion mining," "opinion extraction," "sentiment mining," "subjectivity analysis," "affect analysis," "emotion analysis," and "review mining," but they all serve the same objective. We might be able to figure out what people like or want by evaluating their feelings.



TextBlob

TextBlob is a python library and offers a simple API to access its methods and perform basic NLP tasks.

Textblob's sentiment function returns polarity and subjectivity as two characteristics.

The polarity ranges from [-1,1], with 1 denoting a positive statement and -1 denoting a negative one. Subjective statements usually refer to personal feelings, emotions, or judgments, whereas objective sentences refer to facts. Subjectivity is also a float with a value between 0 and 1.

V. GRADING SYSTEM / RUBRIC (please see separate sheet)

VI. LABORATORY ACTIVITY

Perform the following:

1. Add the following code:

```
from textblob import TextBlob
```

2. Declare a variable that will accept a sentence:

```
sentence = input("Ënter a sentence : ")
```

3. Next initialize TextBlob

```
s = TextBlob(sentence)
```

4. Then check the polarity of the sentence by using the code below:

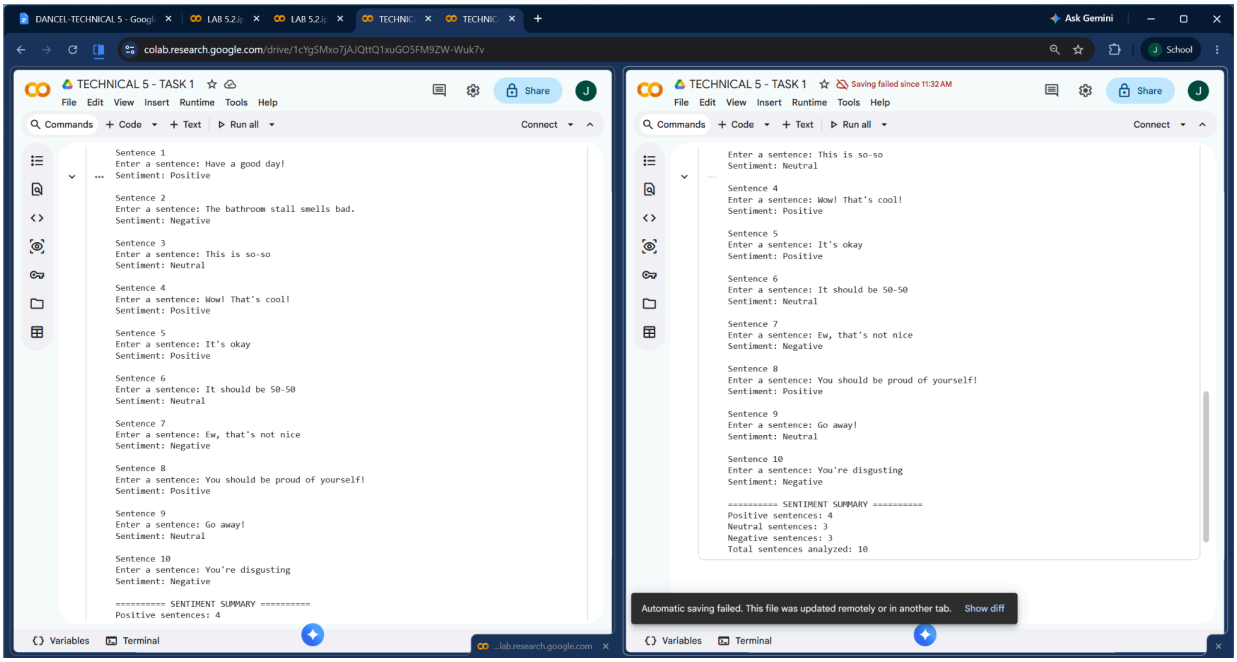
```
p = s.sentiment.polarity
```

5. Make a decision based on the polarity. If the polarity of the sentence is less than zero the sentiment is negative. If the polarity is equals to zero, then the sentiment is neutral. If the polarity is greater than zero, the sentiment is positive.

```
#Negative p<0, Neutral p=0, Positive p>0
if p <0:
print("Negative")
elif p==0:
print("Neutral")
elif p>0 and p<=1:
print("Positive")
```

6. Make the program perform the sentiment analysis 10 times and make a summary of how many sentences are positive, neutral or negative.

OUTPUT:



CODE:

```
!pip install textblob
from textblob import TextBlob

# Initialize counters
positive = 0
neutral = 0
negative = 0

# Perform sentiment analysis 10 times
for i in range(10):
    print("\nSentence", i + 1)

    # Accept a sentence from the user
    sentence = input("Enter a sentence: ")

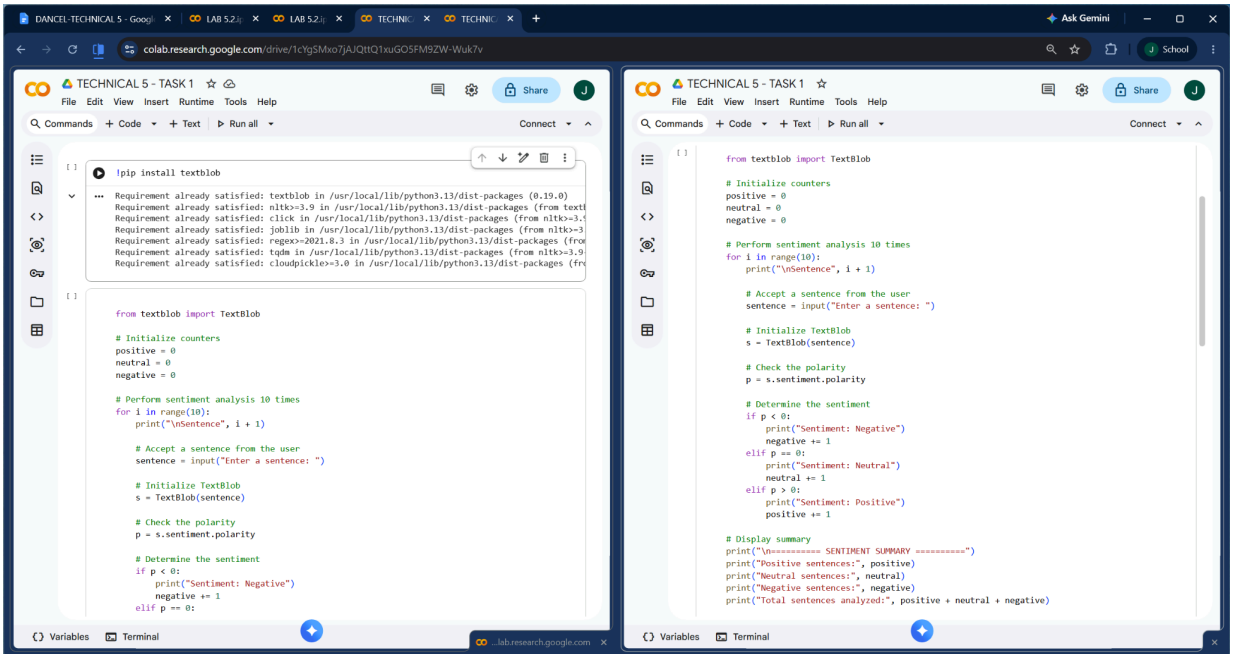
    # Initialize TextBlob
    s = TextBlob(sentence)
```

```
    # Check the polarity
    p = s.sentiment.polarity

    # Determine the sentiment
    if p < 0:
        print("Sentiment: Negative")
        negative += 1
    elif p == 0:
        print("Sentiment: Neutral")
        neutral += 1
    elif p > 0:
        print("Sentiment: Positive")
        positive += 1

# Display summary
print("\n========== SENTIMENT SUMMARY ==========")
print("Positive sentences:", positive)
print("Neutral sentences:", neutral)
print("Negative sentences:", negative)
print("Total sentences analyzed:", positive + neutral + negative)
```

CODE SCREENSHOT:



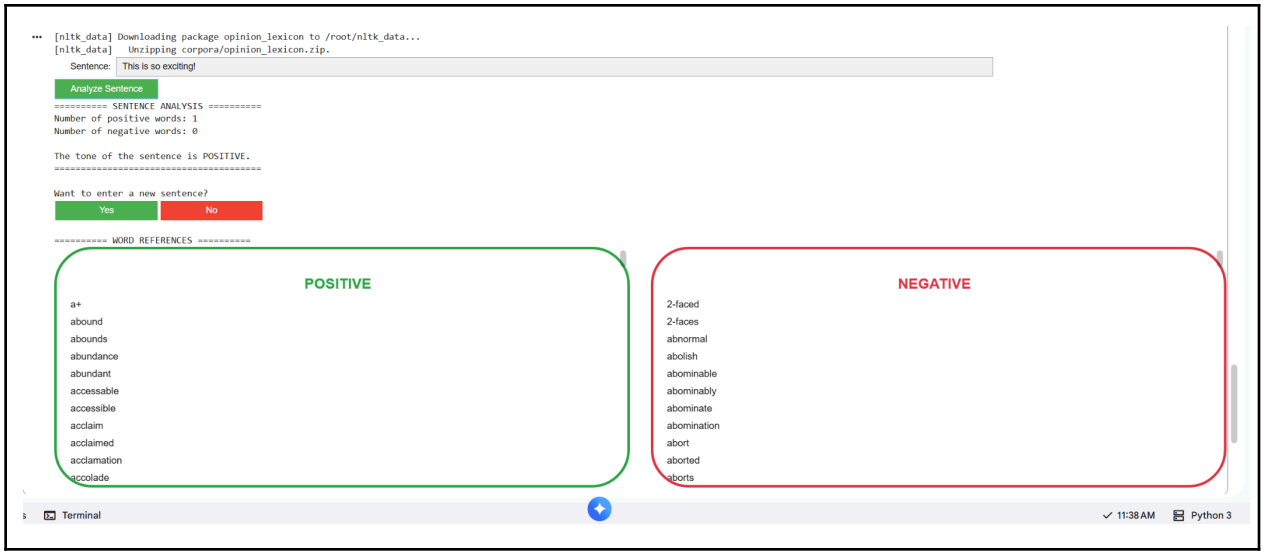
VII. CHALLENGE

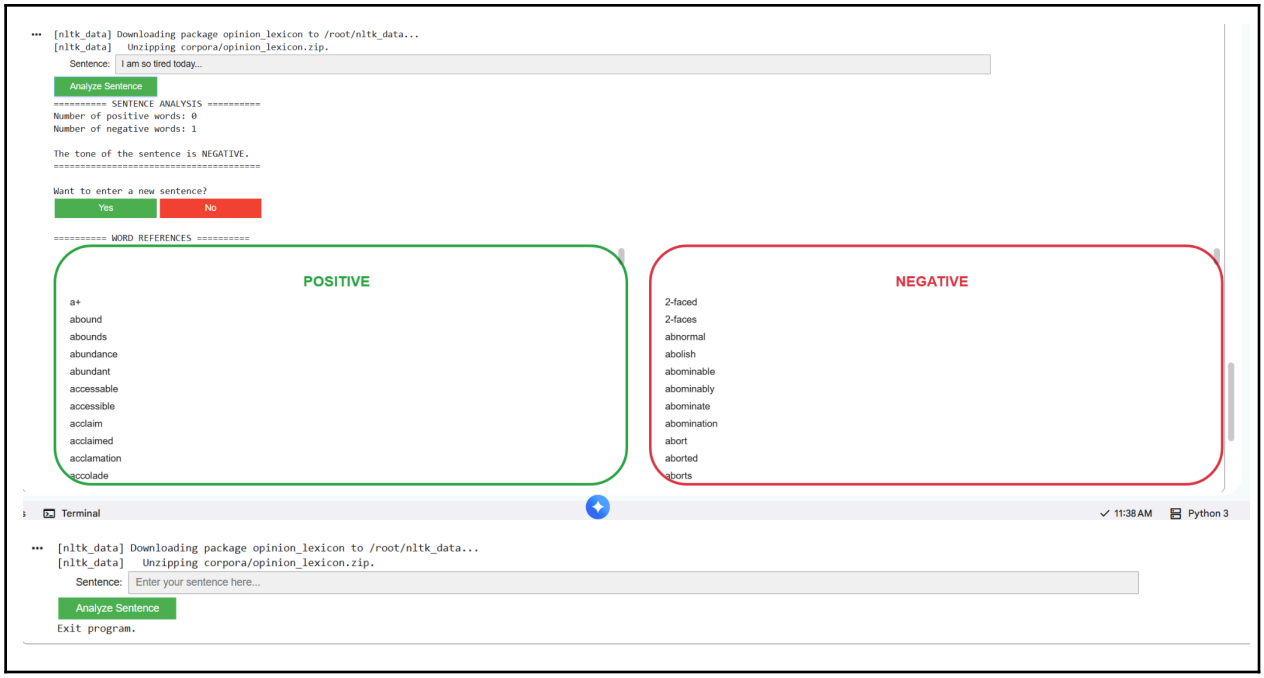
Write a program that will ask the user for a string containing a sentence. The program will then compute the happiness and sadness level of the sentence using the two functions described below. If the happiness level is higher than sadness level, then the tone of the sentence is happy. If the sadness level is higher, then the tone of the sentence is sad. Otherwise, it is neutral.

Find and print the tone of the sentence by printing the number of happy words followed by the number of sad words, followed by a simple statement of the analysis (whether positive, negative or neutral).

To accomplish this, you will write a function called number_happy(sentence) which returns the number of words in a given string called “sentence” that are happy. To do this, look for a public dataset containing a list of happy words. Find the total number of words in “sentence” that are in the happy words dataset.
Next, write a second function called number_sad(sentence) that works the same way but instead counts the number of sad words, also from a separate dataset.

OUTPUT:





CODE:

```
# Import required libraries
import nltk
import re
import html
import ipywidgets as widgets
from nltk.corpus import opinion_lexicon
from IPython.display import display, HTML, clear_output

# Download dataset
nltk.download('opinion_lexicon')

# Get positive and negative words
positive_words = set(opinion_lexicon.positive())
negative_words = set(opinion_lexicon.negative())

# Function to count positive words
def number_happy(sentence):
    words = re.findall(r"[a-zA-Z]+", sentence.lower())
    count = 0

    for word in words:
        if word in positive_words:
```

```
            count += 1

    return count

# Function to count negative words
def number_sad(sentence):
    words = re.findall(r"[a-zA-Z]+", sentence.lower())
    count = 0

    for word in words:
        if word in negative_words:
            count += 1

    return count

# Create input field and buttons
sentence_input = widgets.Text(
    placeholder="Enter your sentence here...",
    description="Sentence:",
    layout=widgets.Layout(width="80%")
)

analyze_button = widgets.Button(
    description="Analyze Sentence",
    button_style="success"
)

yes_button = widgets.Button(
    description="Yes",
    button_style="success"
)

no_button = widgets.Button(
    description="No",
    button_style="danger"
)

output = widgets.Output()
```

```
# Function to display word references
def display_words():

    positive_html = "".join(
        f"<div>{html.escape(word)}</div>"
        for word in sorted(positive_words)
    )

    negative_html = "".join(
        f"<div>{html.escape(word)}</div>"
        for word in sorted(negative_words)
    )

    display(HTML(f"""
    <div style="
        display: flex;
        justify-content: center;
        gap: 30px;
        width: 100%;
        font-family: Arial;
    ">

        <div style="
            flex: 1;
            height: 350px;
            box-sizing: border-box;
            border: 4px solid #28a745;
            border-radius: 55px;
            padding: 20px;
            text-align: center;
            overflow-y: auto;
        ">
            <h2 style="color: #28a745;">POSITIVE</h2>
            <div style="text-align: left; line-height: 1.8;">
                {positive_html}
            </div>
        </div>

        <div style="
```

```
            flex: 1;
            height: 350px;
            box-sizing: border-box;
            border: 4px solid #dc3545;
            border-radius: 55px;
            padding: 20px;
            text-align: center;
            overflow-y: auto;
        ">
            <h2 style="color: #dc3545;">NEGATIVE</h2>
            <div style="text-align: left; line-height: 1.8;">
                {negative_html}
            </div>
        </div>

    </div>
    """))

# Function to analyze the sentence
def analyze_sentence(button):

    sentence = sentence_input.value.strip()

    if not sentence:
        with output:
            clear_output(wait=True)
            print("Please enter a sentence first.")
        return

    happy = number_happy(sentence)
    sad = number_sad(sentence)

    with output:
        clear_output(wait=True)

        # Display analysis
        print("========== SENTENCE ANALYSIS ==========")
        print("Number of positive words:", happy)
        print("Number of negative words:", sad)
```

```
        if happy > sad:
            print("\nThe tone of the sentence is POSITIVE.")

        elif sad > happy:
            print("\nThe tone of the sentence is NEGATIVE.")

        else:
            print("\nThe tone of the sentence is NEUTRAL.")

        print("========================================")

        # Display follow-up question
        print("\nWant to enter a new sentence?")

        display(widgets.HBox([yes_button, no_button]))

        # Display word references
        print("\n========== WORD REFERENCES ==========")
        display_words()

# Function for Yes button
def enter_new_sentence(button):

    sentence_input.value = ""

    with output:
        clear_output(wait=True)
        print("Enter a new sentence above and click Analyze Sentence.")

# Function for No button
def exit_program(button):

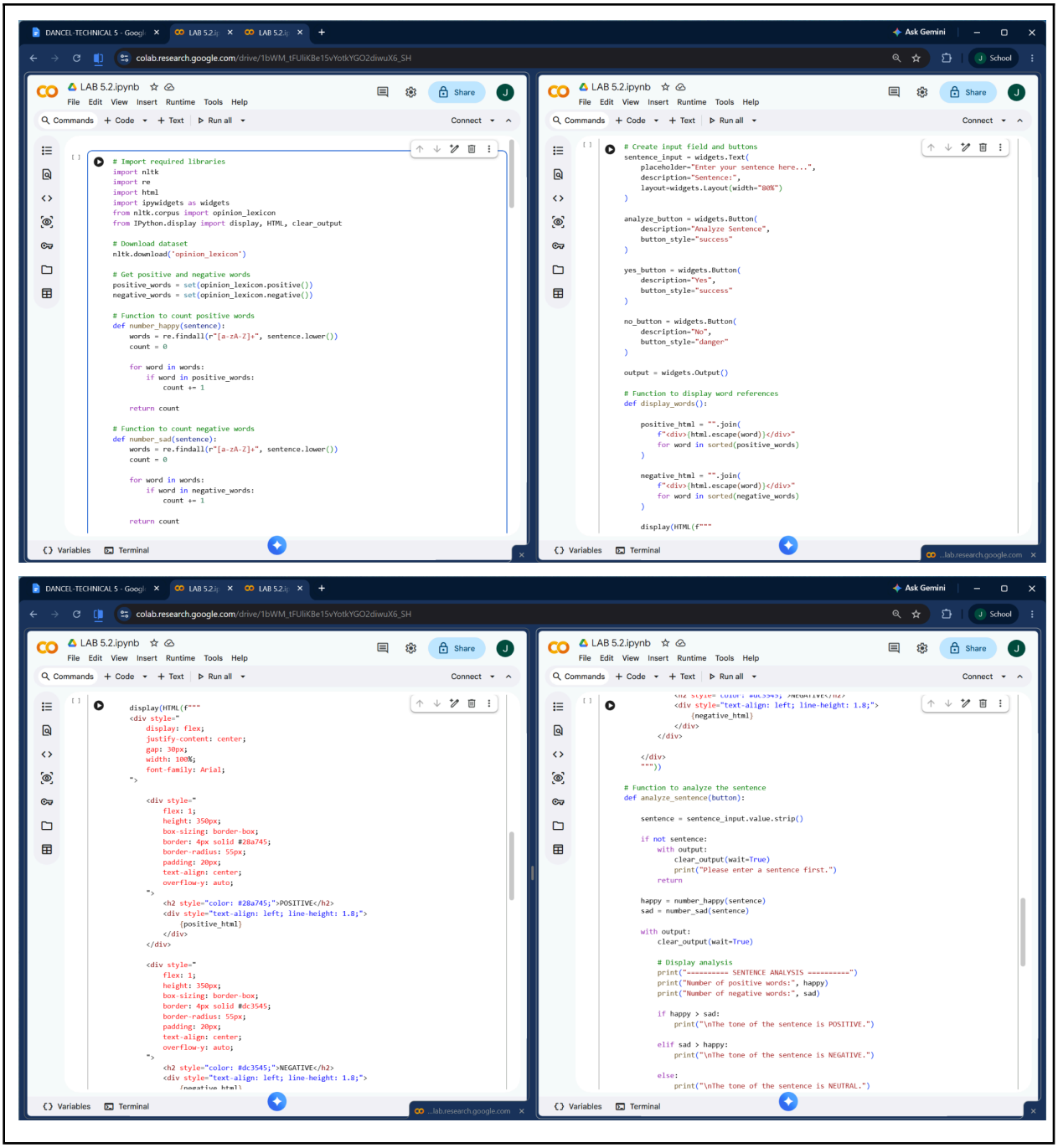
    sentence_input.value = ""

    with output:
        clear_output(wait=True)
        print("Exit program.")
```

```
# Connect buttons to functions
analyze_button.on_click(analyze_sentence)
yes_button.on_click(enter_new_sentence)
no_button.on_click(exit_program)

# Display the program interface
display(widgets.VBox([
    sentence_input,
    analyze_button,
    output
]))
```

CODE SCREENSHOT:


LAB 5.2.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all Connect
Import required libraries
import nltk
import re
import html
import ipywidgets as widgets
from nltk.corpus import opinion_lexicon
from IPython.display import display, HTML, clear_output
Download dataset
nltk.download('opinion_lexicon')
Get positive and negative words
positive_words = set(opinion_lexicon.positive())
negative_words = set(opinion_lexicon.negative())
Function to count positive words
def number_happy(sentence):
words = re.findall(r"[a-zA-Z]+", sentence.lower())
count = 0
for word in words:
if word in positive_words:
count += 1
return count
Function to count negative words
def number_sad(sentence):
words = re.findall(r"[a-zA-Z]+", sentence.lower())
count = 0
for word in words:
if word in negative_words:
count += 1
return count
Variables Terminal
LAB 5.2.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all Connect
Create input field and buttons
sentence_input = widgets.Text(
placeholder="Enter your sentence here...",
description="Sentence:",
layout=widgets.Layout(width="80%")
)
analyze_button = widgets.Button(
description="Analyze Sentence",
button_style="success"
)
yes_button = widgets.Button(
description="Yes",
button_style="success"
)
no_button = widgets.Button(
description="No",
button_style="danger"
)
output = widgets.Output()
Function to display word references
def display_words():
positive_html = "".join(
f" {html.escape(word)} "
for word in sorted(positive_words)
)
negative_html = "".join(
f" {html.escape(word)} "
for word in sorted(negative_words)
)
display(HTML(f"""
Variables Terminal
DANCEL-TECHNICAL 5 - Googl
LAB 5.2.ipynb
colab.research.google.com/drive/1bWM_tFUliKBe15vYotkYGO2diwuX6_SH
Ask Gemini
School
LAB 5.2.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all Connect
display(HTML(f"""
<div style="
display: flex;
justify-content: center;
gap: 30px;
width: 100%;
font-family: Arial;
">
<div style="
flex: 1;
height: 350px;
box-sizing: border-box;
border: 4px solid #28a745;
border-radius: 55px;
padding: 20px;
text-align: center;
overflow-y: auto;
">
<h2 style="color: #28a745;">POSITIVE</h2>

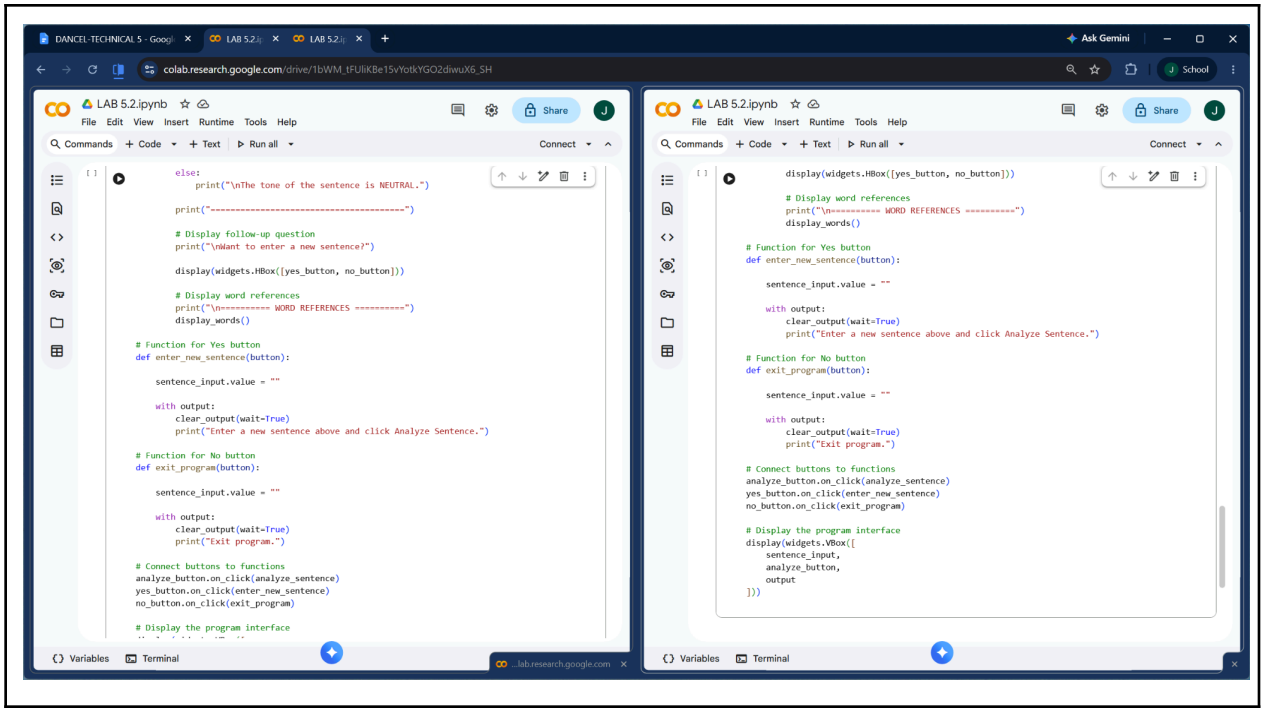
{positive_html}

<div style="
flex: 1;
height: 350px;
box-sizing: border-box;
border: 4px solid #dc3545;
border-radius: 55px;
padding: 20px;
text-align: center;
overflow-y: auto;
">
<h2 style="color: #dc3545;">NEGATIVE</h2>

Variables Terminal
LAB 5.2.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all Connect

{negative_html}

"""))
Function to analyze the sentence
def analyze_sentence(button):
sentence = sentence_input.value.strip()
if not sentence:
with output:
clear_output(wait=True)
print("Please enter a sentence first.")
return
happy = number_happy(sentence)
sad = number_sad(sentence)
with output:
clear_output(wait=True)
Display analysis
print("========== SENTENCE ANALYSIS ==========")
print("Number of positive words:", happy)
print("Number of negative words:", sad)
if happy > sad:
print("\nThe tone of the sentence is POSITIVE.")
elif sad > happy:
print("\nThe tone of the sentence is NEGATIVE.")
else:
print("\nThe tone of the sentence is NEUTRAL.")
Variables Terminal



VIII. REFERENCES

- Artasanchez A. Joshi P.(2020). Artificial Intelligence with Python: You Complete Guide to Building Intelligent apps using Python 3.x 2nd Edition. Packt Publishing
- Attaway, Stormy (2013). MATLAB ®. A Practical Introduction to Programming and Problem Solving Third Edition. Elsevier, Inc.

Online Sources:

- https://www.cuelogic.com/blog/role-of-python-in-artificial-intelligencehttps://monkeylearn.com/sentiment analysis
- https://towardsdatascience.com/a-beginners-guide-to-sentiment-analysis-in-python-95e354ea84f6 • https://research.google.com/colaboratory
- https://www.youtube.com/watch?v=RLYoEyIHL6A